

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	Fixed (→ Fixed.md)	9
Bug	In progress	2
Bug	Operator-blocked	1
Bug	Queued	14
Bug	Ready for testing	1
Task	Completed (→ Fixed.md)	1
Task	In progress	9
Task	Operator-blocked	3
Task	Queued	21
Task	Ready for testing	1
TOTAL		62

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-ho SOCKS routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Jackett + compose env-forward
BOB-068	Bug	In progress	High	RD2-00: unattributed, unreviewed Aut commit mechanism pushing to main
BOB-069	Task	In progress	High	RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape back
BOB-074	Task	In progress	Medium	RD2-07: DDoS-class testing fully absent from the mandated test-type matrix

ATM ID	Type	Status	Severity	Description
BOB-077	Task	Operator-blocked	High	RD2-10: Identify second host running the Auto-commit rsync/sync mechanism (OPERATOR-DECISION)
BOB-078	Task	Queued	High	RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire it
BOB-080	Task	Queued	High	RD2-13: Retroactive Fable-xhigh code review of the substantive Auto-commit diffs
BOB-082	Task	Queued	High	RD2-15: Create BOB-064..067 workable items for the four Lava-porting findings (closes GA-05)
BOB-085	Task	Queued	Medium	RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10)
BOB-087	Task	Completed (→ Fixed.md)	High	RD2-20: Wire docs_chain / commit-sea sync hook per §11.4.106(F) so DB writes cannot land without MD mirror
BOB-088	Task	Queued	Medium	RD2-21: Complete/verify README Tracked-Items + Status Documents table row-completeness (GA-07 remainder)
BOB-090	Task	In progress	Medium	RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack
BOB-093	Task	In progress	High	RD2-28: Live compose bring-up + verification of rustracker ReDoS regex bounds deployment container (closes runtime half of GA-12)
BOB-094	Task	In progress	Medium	RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection
BOB-095	Task	In progress	Medium	RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos for Go-side triangle
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage for exposed download-proxy/merge endpoints (canonical impl of RD2-07)
BOB-100	Task	Queued	Low	RD2-39: Bump submodules/jackett one commit (canonical impl of RD2-09)
BOB-101	Task	Operator-blocked	High	GA-19/RW-09: Is -profile go parity still release goal? (gates RW-10..13) — OPERATOR-DECISION
BOB-102	Task	Operator-blocked	Medium	RW-05: LAN-exposure threat model — tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION
BOB-104	Task		Medium	

ATM ID	Type	Status	Severity	Description
		In progress		§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge
BOB-106	Task	Queued	Medium	§11.4.238 followup: §11.4.84 quiescence check helper for the unattributed auto-commit path
BOB-107	Task	Queued	Medium	§11.4.238 followup: pre-dispatch existence check for subagent task-brief source inputs
BOB-109	Task	Queued	Medium	BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix
BOB-110	Task	Queued	Medium	BOB-074 followup: UX-class test coverage (accessibility/usability) absent
BOB-111	Task	In progress	High	BOB-074 followup: configure real rate limiting for boba's 3 public HTTP endpoints
BOB-114	Task	Queued	Medium	BOB-074 followup: self-validation gold bad fixture for the rate-limit detector
BOB-120	Bug	Queued	Critical	3rd forced-logout incident 2026-08-18 23:45:49 — SIGKILL user@1000 + preventive-timer-inside-user-slice architectural gap
BOB-121	Task	Ready for testing	Important	External watchdog for the forced-logout architectural gap (task #85, incident #)
BOB-129	Bug	Fixed (→ Fixed.md)	Medium	Potential production slowapi/starlette defect flagged by Task 105 subagent
BOB-131	Bug	Fixed (→ Fixed.md)	Medium	qbittorrent-proxy podman common crash pre-existing, surfaced during BOB-129 investigation
BOB-135	Bug	Queued	Low	Test isolation: test_list_hooks_after_create fails in bulk suite (Permission denied / config)
BOB-137	Bug	In progress	High	Merge service on 7187 wedges while the same process still serves 7186 (GIL starvation by one spinning thread)
BOB-141	Task	Queued	Low	CLAUDE.md claims the Go profile serves 7186/7187/7188 but its container binds only 7187 — doc contradicts the Dockerfile
BOB-143	Bug	Queued	Medium	Orphaned .worktrees/ dirs (46M, unresolvable gitdir) pollute gate scan scope and manufacture false BOB-126 class findings
BOB-144	Bug	Fixed (→ Fixed.md)	Low	/theme/stream calls the disconnect procedure unguarded — fail-closed but via an

ATM ID	Type	Status	Severity	Description
				uncaught traceback, inconsistent with two SSE generators
BOB-145	Bug	Fixed (→ Fixed.md)	High	Fix the 7187 wedge: offload and/or memoise Deduplicator.merge_results s O(N^2) regex work stops blocking the asyncio event loop
BOB-146	Bug	Fixed (→ Fixed.md)	High	Constitution §11.4.252 detector undercounts by 29% (30 vs 42 AST gro truth) — 4 distinct blind spots make its output a floor, not a census
BOB-148	Bug	Fixed (→ Fixed.md)	Medium	Standing red unit test nothing tracked test_no_credentials asserts has_session False, gets True — real defect or non-hermetic test, undecided
BOB-149	Bug	Queued	Low	Managed-plugin count diverges 43/42/ across constitution, CLAUDE.md and t README badge; the badge is hand-maintained and unguarded
BOB-150	Task	Queued	Low	pre_build_verification.sh invariant label read N/44 but only 35 invariants are labelled
BOB-151	Task	Queued	Medium	CM-SCRIPT-DOCS-SYNC is a named g with no implementation, and 24 of 36 scripts have no companion doc
BOB-152	Bug	Queued	Medium	Constitution sweep walks vendored thi party code: 82% of one gate's 38,291 findings come from submodules/
BOB-153	Bug	Fixed (→ Fixed.md)	Medium	Go profile cannot build: go.mod require go 1.26.2 but the Dockerfile builder is golang:1.23-alpine
BOB-154	Bug	Ready for testing	Medium	Host venv and production container run different starlette versions (1.4.1 vs 1.
BOB-155	Bug	Fixed (→ Fixed.md)	High	workable-items diff reports 'DB and Markdown are in sync' having opened Markdown files when -issues/-fixed are omitted
BOB-156	Bug	Queued	Medium	BOB-145 event-loop regression guard load-sensitive and flaky: 8786ms under host load vs a 900-1500ms ceiling calibrated on a quiet host
BOB-157	Bug	Fixed (→ Fixed.md)	High	Our own BOB-137 stall watchdog can segfault the merge service: faulthandle dump_traceback(all_threads=True) hit unpatched CPython 3.12 defect
BOB-158	Bug	Queued	High	tests/conftest.py cannot run on the production interpreter: binds

ATM ID	Type	Status	Severity	Description
BOB-159	Bug	Queued	Medium	asyncio.events._get_event_loop_policy, 3.13+ private API, while production is 3.12.13 Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:01:17Z Reported-By: T041 independent review (IMPORTANT-2), partially remediated What (the report, verbatim): The -recreate path was fixed this round: run_ownership_precondition -> stack_down -> run_ownership_repair -> stack_up, so the repair walks a tree no container can write to. That ordering is now asserted behaviourally by three new checks in tests/unit/test_start_reload_recreate.sh, each killed by its paired 1.1 mutation (11/11 RED-). The WARM path is NOT covered by this and this item tracks the remainder. On warm ./start.sh the gate runs before the invocation brings anything up, but if the stack is ALREADY running, containers can write while the repair walks. A container can then create a new non-operator-owned file BEHIND the walk, after which the completion marker records completion of a tree that is not, and those stragglers never repaired without a manual -force BOUNDED, not dismissed: after any successful pass the marker is valid for scope fingerprint and the repair short-circuits without walking (marker_is_valid so the window requires a stale-or-absent marker AND a live stack together. Declaring a new scope entry re-arms the marker, which is exactly how that combination arises in practice. NOT DONE THIS ROUND, with the reason stated rather than hidden: the clean guard is cheap marker-only probe mode on script ownership_repair.sh that answers has-scope-been-repaired without walking the tree. The existing -dry-run cannot serve deliberately SKIPS the marker fast-path (FORCE=0 && DRY_RUN=0 guard) and always walks, so using it as a warm-stack probe would walk the tree twice on every start. Adding that mode requires editing ownership_repair.sh, which another staff was actively editing during this round;

ATM ID	Type	Status	Severity	Description
BOB-160	Task	Queued	Medium	duplicating <code>marker_is_valid</code> into <code>start.sh</code> instead would be a near-identical fork (11.4.251). Deferred to this item rather than done unilaterally or silently. The misleading comment at the warm-start site (which claimed the gate runs before any container writes) has been corrected to state this boundary honestly. Affect scope / file-scope manifest: <code>start.sh</code> (warm-start dispatch, <code>run_ownership_gate</code> call site); <code>scripts/ownership_repair.sh</code> (marker fast-path) Reproduction / context: 1. Ensure the stack is UP. 2. Change <code>config/owned_paths.yaml</code> so the scope fingerprint changes (this re-arms marker by design, data-model E2). 3. Run a warm <code>./start.sh</code> (no <code>-recreate</code>). 4. The ownership repair walks and chowns the declared tree while containers are still running and able to write into it. Acceptance criteria: A warm <code>./start.sh</code> cannot complete an ownership repair while a container that writes to a declared location is running. Either the repair is deferred with an actionable refusal name <code>-recreate</code>, or the stack is quiesced for a walk. Asserted behaviourally in <code>tests/unit/test_start_reload_recreate.sh</code> alongside existing <code>PRECONDITION_BEFORE_DOWN / REPAIR_AFTER_DOWN / REPAIR_BEFORE_UP</code> checks, each with a paired 1.1 mutation that kills it. Reported-Via: §11.4.202 reporting directive task on 2026-08-21T19:02:48 Reported-By: AI What (the report, verbatim): Independent §11.4.209 review (IMPORTANT-3) found two of the features strongest automated checks were never executed by any runner: (1) <code>tests/ownership/test_container_writes_owned_files.py</code> – the §11.4.115 RED-turned-regression-guard for FR-011/FR-007, proven via a <code>docker-compose.yml</code> revert mutation – was moved out of <code>tests/integration/</code> (commit 58d340a, to escape an autous fixture hang) but no runner (<code>ci.sh</code>, <code>test-run-all-tests.sh</code>) was ever extended to cover <code>tests/ownership/</code>. (2) <code>tests/pre_b</code>

ATM ID	Type	Status	Severity	Description
BOB-161	Task	Queued	High	test_.sh, the §1.1 paired-mutation meta-tests for the scripts/pre_build/check_cm_.sh gate family, was executed by NOTHING — self-reported honestly in commit 04742d7’s own message (‘STILL OPEN (reported, not fixed): tests/pre_build/ is executed by NOTHING’) but never tracked as a workable item (a §11.4.197 loss-of-requirements gap) and never wired into scripts/pre_build_verification.sh invariant 30, which globbed only tests/unit/test_.sh. Affected scope / file-scope manifest: ci.sh; scripts/pre_build_verification.sh invariant 30 (CM-BASH-UNIT-TESTS-EXECUTED) Reproduction / context: grep -rn test_container_writes_owned ci.sh test.sh run-all-tests.sh scripts/ returned zero runner hits (only docs/spe mentioned the path); grep in scripts/pre_build_verification.sh showed invariant 30’s for-loop globbing only tests/unit/test_.sh, never tests/pre_build/test_.sh Acceptance criteria: ci.sh gains a runtime-gated pytest tests/ownership/ stage (skips honestly, rc=0, when no podman/docker present); scripts/pre_build_verification.sh invariant 30’s glob covers both tests/unit/test_.sh and tests/pre_build/test_*.sh, verified by an extracted standalone run of the invariant exact block reporting a non-zero RAN count that includes files from both directories. Reported-Via: §11.4.202 reporting directive task on 2026-08-21T19:31:12Z Reported-By: BOB-092 remediation, residual finding What (the report, verbatim): §11.4.69 names CM-NO-FAIL-OPEN-SKIP as one of three mandatory build gates: it audits sink-side probe helpers and FAILs if any code path converts an empty or unreachable response into a PASS-counting SKIP for feature class with a sink-side probe. The project does not have it. Why this matter now rather than in the abstract: the BOB-092 remediation just removed TW fail-opens of exactly this class from tests/e2e/test_live_stack_evidence.py — the

ATM ID	Type	Status	Severity	Description
				nnmclub SKIP-on-404 (already removed 7baef2b) and a surviving sibling at test_iporrents_is_authenticated_in_sea that skipped on 'not authenticated and status != success' while asserting 'treas as transient outage'. That assertion was false for rejected credentials (upstream_http_403) and for a broken container (plugin_env_missing), both of which are definitive product failures that product already classifies via error_type. Two fail-opens of one class, found one time, is the §11.4.146 extend-to-all-cases signal and the §11.4.238 coverage-escape signal together: the regime did not survive either, an agent reading code did. The remediation's own guards are AST-structural and lethal under three discriminating mutations, but they LIVE THE FILE THEY GUARD, so deleting the file evades them entirely. A pre-build guard is the durable home because it audits the corpus rather than one file. Honest boundary (§11.4.6): this item does NOT claim the two remediated fail-opens are unguarded — they are guarded, with runtime evidence (13 passed in 97.36s against the live stack). It claims the CI has no corpus-wide detector, so the new instance in a different file is invisible again. Affected scope / file-scope manifest: scripts/pre_build/ (gate abs tests/e2e/test_live_stack_evidence.py (guards currently live inside the file the guard) Reproduction / context: grep 'CM-NO-FAIL-OPEN-SKIP' scripts/ test returns exactly ONE hit, and it is tests test_live_stack_evidence.py — the file the guards live in, not a gate. Control needs the same query for CM-OWNERSHIP-INVARIANTS (a gate that does exist) returns 3 files, so the instrument can see gate tokens and the single hit is a real absence, not a blind zero (§11.4.201(7)). Acceptance criteria: A scripts/pre_build gate named CM-NO-FAIL-OPEN-SKIP exists, is wired into scripts/pre_build_verification.sh, and audits side probe helpers for code paths converting an empty/unreachable/error

ATM ID	Type	Status	Severity	Description
BOB-162	Task	Queued	Medium	response into a PASS-counting SKIP for feature class that HAS a sink-side problem ships a golden-TRUE fixture (a real failure open -> gate FIRES) and a golden-FALSE with-carrier (an honest topology/geo shape) and a comment merely MENTIONING phrase -> gate MUST NOT fire), per §11.4.107(10)/§11.4.201(1). Paired §11.4.202 mutation makes the gate FAIL before the gate is trusted. Reported-Via: §11.4.202 reporting directive task on 2026-08-21T19:35:02Z Reported-By: BOB-106/BOB-107 remediation, residual What (the report verbatim): The TESTS for both guards now executed: pre-build invariant 30's glob was extended from tests/unit + tests/pre_build to also cover tests/hooks. MEASURED before/after: the glob expanded to 27 suites with 0 under tests/hooks while 3 existed there; it now expands to 30 with 3 under tests/hooks three pass. But the GUARDS themselves are still invoked by nothing. check-brief inputs.sh is honestly conductor-run rather than automatic: a brief's required-input list lives in free-form prose that the Agent tool's structured input does not expose an automatic prompt-scraping gate would itself be an unproven pattern-match. It seam is the dispatch procedure, and the a documentation + habit change, not a hook. unattributed-commit-guard.sh is different and is why this item exists. R against real history it names 14 violating commits since the last tag (and 21 barrier Auto-commit subjects exist across all r Wiring it into the pre-build or commit seam AS-IS would therefore refuse even commit immediately, on pre-existing data nobody can fix in the moment. That is precisely the 11.4.224(E) brownfield-adoption shape, and 11.4.66/11.4.122 the choice with the operator, not with the agent inventing a ratchet. The options stated so the decision is a decision and a default: (a) immediate hard floor - the seam refuses until all 14 are attributed one-time monotone-decrease ratchet (11.4.135 pattern) - snapshot 14 as the

ATM ID	Type	Status	Severity	Description
BOB-163	Bug	Queued	Medium	baseline, the count may fall and may n rise, so day one is green and the disea cannot spread; (c) changed-commits-on gate only commits created from now o with a scheduled deadline for the historical 14; (d) report-only for a state period, then escalate. Recommendation with the reasoning rather than just the pick: (b). It is the pattern this constitut already uses for exactly this situation, makes the debt visible and bounded immediately, and it cannot be satisfied deleting the guard's name (11.4.227 cl that gaming channel). But it is the operator's call and this item is not clos until that answer is recorded. Honest boundary (11.4.6): this item does NOT claim the 14 commits are harmful in themselves - it claims their ATTRIBUT is absent and that nothing currently prevents the 15th. Affected scope / fi scope manifest: scripts/hooks/ unattributed-commit-guard.sh; scripts/ hooks/check-brief-inputs.sh; scripts/ pre_build_verification.sh; scripts/comm push-all.sh Reproduction / context: l guards run correctly by hand and are covered by passing tests (9/9 and 15/1 each proven non-bluff against a no-op stub). Neither is invoked by scripts/ pre_build_verification.sh or scripts/ commit-push-all.sh, so neither exerts standing detection pressure (11.4.226: guard with no execution seam is not coverage; registration is not coverage) Acceptance criteria: Each guard is invoked by a named seam, OR carries registered deferral pointing at this item For unattributed-commit-guard.sh specifically, the operator has answered adoption question below and the answe recorded as consumer DATA - never an invented ratchet. Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:41:08Z Reported-By: BOB-114 remediation, p existing defect surfaced by BOB-111 landing a real limiter What (the repo verbatim): PRE-EXISTING, NOT INTRODUCED - and proven so rather

ATM ID	Type	Status	Severity	Description
				asserted: git diff shows assertion (c) is BYTE-IDENTICAL to HEAD, and the fact reproduces against the unmodified HEAD script. What changed is not the challenge but the SYSTEM: BOB-111 appears at least partially delivered, because :718 now returns real 429s where it previously returned none. :7185 and :7189 still produce zero 429s. This is a 11.4.248-c corrosion risk and that is why it is filed rather than left as a footnote: run_all_challenges.sh will now fail INTERMITTENTLY, and an intermittent red trains everyone to re-run until green at which point a real regression is dismissed as 'probably the flaky rate-limiter one'. THE DECISION, which is an open call under 11.4.66 and which I deliberately did NOT invent: Does a 429 from a WORKING limiter count as the sibling endpoint being RESPONSIVE? (a) YES 429 proves the endpoint is alive and correctly protecting itself. Assertion (c) accepts 2xx OR 429, and only a connection failure / 5xx / timeout counts as degradation. Risk: a genuinely wedged endpoint that happens to answer 429 would pass. (b) - keep requiring 2xx, but make the challenge limiter-aware: drain or wait the window before the sibling probe (rather than after is served, so the budget is known rather than guessed). (c) Probe sibling a path the limiter exempts (a healthz-check route), so the isolation question is asked without spending the bucket. Recommendation with reasoning, not just a pick: (a) composed with a bounded liveness check - a 429 carrying a well-formed Retry-After IS evidence of a live, correctly-behaving service, and (b) make the challenge slower and couples it to window value that will drift. But the semantics of 'responsive' here are a product judgement, so the answer is recorded, not assumed. RELATED, stated as fact rather than folded in: the detection counts 429 only, not 503, though the specification header says '429 (or equivalent)'. Counting 503 would collide with the crash detection 5xx tally. Worth deciding when BOB-111

ATM ID	Type	Status	Severity	Description
				lands limiters on :7185 and :7189. Affected scope / file-scope manifest challenges/scripts/ ddos_resilience_challenge.sh assertion cross-endpoint isolation; run_all_challenges.sh which invokes it Reproduction / context: Assertion (c) requires a sibling-endpoint probe to answer ^2 (a 2xx). :7187 now enforces real limiter (measured live: x-ratelimit: limit: 120, x-ratelimit-remaining: 86, re after: 45, server: uvicorn). A full challenge run sends roughly 250 requests to :7187 so sibling probes fired during the OTH endpoints' tiers land on an exhausted bucket and receive 429 - which asserts (c) scores as 'endpoint degraded'. Time dependent: one run measured PASS=5 FAIL=2; the UNMODIFIED HEAD scri against the same live stack ~30s later measured PASS=7 FAIL=0 SKIP=2. Acceptance criteria: The operator has answered the classification question because the answer is recorded as consumer D. and assertion (c) implements it. The challenge then produces the same version across 3 consecutive runs against a real limited stack (11.4.50 deterministic consistency), and the fix ships a paired mutation proving assertion (c) still catches a genuinely degraded sibling endpoint narrowing it must not blind it (11.4.201(1)). Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:56:53Z Reported-By: BOB-110 UX-class coverage, discovered by the new axe-core suite on its first live run What (the report, verbatim): This is a REAL user-facing defect, not a test-tuning artifact and it was found by the automated regression rather than by a human squinting at the page - which is exactly the 11.4.238 posture the project is aiming for. The static-grep oracle would NOT have found it. Measured: the served root ships an empty pre-hydration, so any check reading the raw HTML audits a page nobody sees. The violation only exists in the hydrated DOM, which is why 11.4.170 requires
BOB-164	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				rendered oracle and forbids value-equality assertions as the proof a UI is correct. Severity reasoning, stated rather than assumed: this is user-visible and affects the primary dashboard heading, but it degrades legibility rather than breaking function, and the surface is operator-facing rather than public. Medium, not High. The failing test was left FAILING purpose (11.4.238) instead of silenced marked xfail. tests/ux/ currently reports failed, 16 passed; that 1 is this defect. Anyone reading a red UX suite should see it as this item, not as flakiness – and when this is fixed the suite goes fully green, which is the signal that it is closed. HONEST SCOPE LIMIT (11.4.6): only the dashboard landing view was scanned. The /jackett/* sub-routes and the ng-se-hosted :4200 route set were NOT audited. The commands to close both are recorded in docs/testing/ux_accessibility.md. So the item's 21 nodes are a floor, not a total. Affected scope / file-scope manifest: the Angular dashboard served at http://localhost:7187/ (same compiled SPA as production frontend/); production component CSS test files Reproduction / context: Run tests/ux/test_live_dashboard_accessibility.py against the running merge service. axe-core v4.13.0, scanning a real Playwright-rendered JS-hydrated DOM, reports color-contrast violations on 21 nodes. Measure pairs: .brand / h1 text #9d001e on background #3c3f41 = 1.43-1.62:1 (WCAG AA large-text floor is 3:1); tagline #800000 on #3c3f41 = 2.68:1 (body-text floor is 4.5:1). Acceptance criteria: axe-core reports ZERO color-contrast violations against the live rendered dashboard, with the fix made in production component rather than by relaxing the assertion or excluding the rule (11.4.120: reconcile the correct mechanism, never weaken check). The existing tests/ux/ suite is guarded and already fails today, so the R is captured – closure requires it flipping GREEN against the live surface, which is runtime-class evidence per 11.4.226.

ATM ID	Type	Status	Severity	Description
BOB-165	Bug	Queued	High	Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T20:00:08Z Reported-By: surfaced while capturing closure evidence for BOB-092; diagnosis rather than worked around What (the report, verbatim): This is a STALE-AFTER-INTERPRETER-UPGRADE defect, not a missing package. The package is installed; its compiled half is built for the previous interpreter. That distinction matters because 'pip install rpds-py' still gives advice that can appear to succeed while leaving the 313 artefact in place. WHY IT WAS NOT NOTICED EARLIER, stated as fact: the paths that DO work all avoid system python3. ci.sh selects an interpreter through _select_python; the long-running suites in this session ran .venv/bin/python3 -m pytest and passed. So the automated regime is green while the DOCUMENTED operator command is broken - an 11.4.238-shaped gap, since the escape was found by an agent capturing evidence rather than by the regime. WORKAROUND (measured, not theorised): PYTEST_DISABLE_PLUGIN_AUTOLOAD with the needed plugins passed explicitly (p pytest_timeout ...) avoids the schemathesis autoload that drags in jsonschema -> referencing -> rpds. This is a workaround, NOT the fix: it silences import path rather than repairing the install, and it will surprise the next person who runs the documented command verbatim. RELATED, and worth deciding together rather than twice: BOB-154 wants .venv rebuilt on 3.12 to match the container (which runs CPython 3.12.13) while both system and venv run 3.14.6. There are therefore THREE interpreters in play. Whoever rebuilds the venv should confirm rpds resolves for the TARGET interpreter afterwards, or this same closure reappears one directory over. HONEST BOUNDARY (11.4.6): this item does NOT claim any test is wrong or any product code is broken. The product and the venv run suites are unaffected. What is broken is the documented entry point. Affected scope / file-scope manifest: host use

ATM ID	Type	Status	Severity	Description
BOB-166	Bug	Queued	High	site-packages (~/.local/lib/python3/site-packages/rpds/); every CLAUDE.md documented 'python3 -m pytest ...' invocation; any tooling that uses system python3 rather than .venv/bin/python3 Reproduction / context: python3 -m pytest tests/e2e/test_live_stack_evidence.py -q -import-mode=importlib -> ModuleNotFoundError: No module named 'rpds.rpds', raised during collection via schemathesis plugin autoload -> jsonschema -> referencing. core -> rpython MEASURED root cause: system python3 3.14.6, but ~/.local/lib/python3/site-packages/rpds/ ships rpds.cpython-313-x86_64-linux-gnu.so - an extension built for 3.13. rpds/init.py does 'from .rpds import *', and a 313-tagged .so is not importable by 3.14, so the submodule genuinely does not exist for this interpreter. The venv is CORRECT and unaffected: .venv/lib64/python3/site-packages/rpds/ ships rpds.cpython-314-x86_64-linux-gnu.so and '.venv/bin/python3 -c import rpds' succeeds. Acceptance criteria: python3 -m pytest tests/unit/ -import-mode=importlib - the command CLAUDE.md documents - reaches collection without ModuleNotFoundError OR CLAUDE.md is corrected to document the supported runner explicitly. Either the documented command and the working command agree (11.4.99: a goal that misleads is the documentation-laying equivalent of a PASS-bluff). WHAT. §11.4.19 requires closure migration to be ATOMIC: a resolving item moves from Fixed.md, DISAPPEARS from Issues_Summary (open-only) and APPEARS in Fixed_Summary (closed-only). Ten rows violate all three properties right now — they carry a terminal status yet current_location='Issues', so they render in docs/Issues.md and are listed in docs/Issues_Summary.md with a status that literally reads '(→ Fixed.md)', while appearing 0 times in Fixed_Summary.md. Any reader of the canonical open track told these are open work. MANIFEST

ATM ID	Type	Status	Severity	Description
				(measured 2026-08-21): BOB-087 (Completed), BOB-129, BOB-131, BOB-144, BOB-145, BOB-146, BOB-147, BOB-153, BOB-155, BOB-157 (all 'Fixed (→ Fixed.md)'). Confirmed rendering: BOB-155/087/157 each grep 1 in Issues.md, 0 in Fixed.md, present in Issues_Summary.md, 0 hits in Fixed_Summary.md. ROOT CAUSE (reproduced at runtime, not inferred). 'close' performs the atomic migration and REQUIRES -evidence. 'update -status' any §11.4.15 closed-set value with NO evidence and NO migration — and it knows the location, because it prints it a COPY of the real DB: \$ workable-items update -id BOB-065 -db repro.db -status 'Fixed (→ Fixed.md)' update: BOB-065 updated in Issues (status=Fixed (→ Fixed.md), type=Task) \$ sqlite3 repro.db 'SELECT atm_id,status,current_location FROM Issues WHERE atm_id=BOB-065 Fixed (→ Fixed.md) Issues Summary' seam that is supposed to be the ONLY closure path (close, evidence-gated per §11.4.146(D3)) has a parallel unguarded path that reaches the same status while skipping BOTH the evidence requirement and the migration. WHY NOTHING CAUGHT IT (§11.4.238 coverage-escape audit). Three standing checks stay GREEN on a DB holding the forbidden row, verified on the poisoned copy from the repo root: no cwd artifact is involved: (1) 'validate' 'OK — 164 items, all invariants satisfied (it has no status↔location coherence invariant); (2) 'diff' → 'in sync' (DB and Markdown agree — on the WRONG status agreement is not correctness); (3) CLOSURE-SEAM-BINDS CHECK A → I (it flags NON-terminal rows whose id appears in a work commit; these rows are terminal, so they are outside its prediction by construction). This was found by reading a status tally, not by the automated regime — a §11.4.238 discovery-channel escape, which is itself a defect of equal standing to the mis-location of rows. ACCEPTANCE. (a) 'update' REFUSES a terminal status and names 'close' as the correct path, with a paired

ATM ID	Type	Status	Severity	Description
				§1.1 mutation proving the refusal (removing the guard must make the mutation pass). (b) ‘validate’ grows a status↔location coherence invariant that FAILS on the forbidden state, with a golden-bad fixture and a negative context (a legitimately terminal row in Fixed mode NOT fire — §11.4.201(1)). (c) The 10 existing rows are drained to Fixed with class-matched evidence per row, or, when a row’s evidence cannot be produced, honestly re-opened rather than migrating on a bare assertion. (d) Honest boundaries this closes the update-path hole and the detection gap; it does not claim every historical status write was evidence-backed. NOT CLAIMED. No fix is implemented by this filing. The 10 rows are untouched; draining them is acceptance (c) and each needs its own evidence, not a bulk UPDATE. WHAT. download-proxy exposes two Server-Sent-Events routes. They are the same expensive class — long-lived connections that hold a worker and a generator for their lifetime — but only is rate-limit classed: routes.py:801 <pre>@router.get('/search/stream/{search_id}') @_rl('sse_stream') <- classed routes.py:150 @router.get('/theme/stream') async def stream_theme(...) <- NO limiter decorator _rl(cls) resolves to limiter.limit(limit_for(cls)), so /search/stream is bound to the sse_stream class while /theme/stream falls through to the application default. Measured live on the running stack: /api/v1/theme/stream reports x-ratelimit-limit 120. PROVENANCE + A CORRECTION WORKING KEEPING (§11.4.6). This was surfaced by the BOB-109 scaling agent, whose report framed it as: ‘sse_stream_limit_decorator is defined and imported/applied nowhere ... SSE falls through to the 120/minute default: 24x less protected’. That framing is WRONG and was NOT filed as given. The agent searched for one symbol NAME the wiring uses a different mechanism (@_rl('sse_stream')), and it IS applied to /search/stream. Verified by reading </pre>
BOB-167	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
BOB-168	Task	Queued	Low	routes.py:795-806 and the _rl helper a routes.py:46-51, with a control needle confirming other *_limit_decorator sym show real usage in api/init.py so the z hit was not a blind search. The agent's MEASUREMENT (120 on /theme/stream) was correct and is what makes this a r finding; its MECHANISM was not. Bot halves are recorded so the next reader does not re-derive the same wrong cau WHY IT MATTERS. An unclassified SSE endpoint is the cheapest way to pin se resources: each connection is held ope and the default class permits 120/min them. The declared sse_stream class e precisely because this route shape nee tighter bound than ordinary GETs. ACCEPTANCE. (a) A decision, recorded whether /theme/stream belongs in the sse_stream class or genuinely warrant default — this is a policy question, not automatically a bug to patch. (b) If it belongs in sse_stream, the decorator is applied and a test drives BOTH SSE ro and asserts each returns its INTENDE class limit from x-ratelimit-limit, so a future route added without a class is caught. (c) A guard that enumerates S shaped routes and fails on any that can no explicit rate-limit class — the gener form, so the third SSE route does not repeat this. (d) Honest boundary: this not claim 120/min is exploitable in this deployment; with network mode host a no reverse proxy every caller shares th 127.0.0.1 bucket, which BOB-111 measured and recorded separately. NO CLAIMED. No change made. The limit values were read from headers, never driven to exhaustion — the limiter is p and shared with concurrent agents on host (§11.4.119). WHAT. scripts/run_all_challenges.sh:60 lists “scaling_horizontal_challenge.sh” its challenge set. challenges/scripts/scaling_horizontal_challenge.sh does n exist: \$ sed -n ‘66p’ scripts/run_all_challenges.sh “scaling_horizontal_challenge.sh” \$ ls challenges/scripts/

ATM ID	Type	Status	Severity	Description
				scaling_horizontal_challenge.sh ls: can't access ...: No such file or directory VERIFIED independently, not taken on report — surfaced by the BOB-109 scaling agent and re-checked here by direct invocation. WHY IT MATTERS. Whether this is cosmetic or a §11.4.201 gate-honesty defect depends entirely on how the runner treats a missing entry, and is the first thing to determine: if it SKIPS silently, the challenge bank advertises coverage it does not have (a §11.4.266 claim-vs-reality row with no passing challenge behind it); if it FAILs, the runner is permanently red for a reason unrelated to the system under test, which trains readers to ignore it. Neither outcome is acceptable; they need different fixes. ACCEPTANCE. (a) Determine and record the runner's actual behaviour on the missing entry by invoking it, not by reading it. (b) EITHER author the challenge, OR remove the entry — with §11.4.124 discipline: check git history whether it once existed and was deleted since a silently-dropped challenge is the more interesting defect. (c) If the runner silently skips missing entries, that is its own finding: a missing challenge must be loud (§11.4.3 SKIP-with-reason at minimum), never absent-and-quiet. SEVERITY. Low as a defect, but it sits on the challenge-coverage seam, so (c) must deserve its own item. WHAT. The §11.4.65 markdown-export mandate requires every in-scope doc to ship .html/.pdf twins. 286 of the 326 .html files under docs/ (excluding dist/ and node_modules/) are pandoc FRAGMENTS — they begin at WHAT. The BOB-109 scaling suite gates two timing tests on loadavg_1m/nproc at 0.75 — on this 8-cpu host, a 1-minute load average at or below 6.0 — and SKIPS loudly otherwise. That gate is correct as was adopted for a good reason (§11.4.201(8)): the author validated a 2nd growth-exponent threshold, then observed it FALSE-FAIL correct code at load 11.1 (baseline span 2.136), and at load 15-2
BOB-169	Bug	Queued	Medium	
BOB-170	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				measured baseline spans of 2.358 against injected-cubic mutants at 2.278-2.331 — the signal is smaller than the noise and the two are not separable in either direction. Gating was the honest response. THE PROBLEM. The gate has consequently NEVER RUN under its own precondition. Every recorded run today measured 9.15-23; the independent reviewer measured 9.66 during review; the conductor measured 6.45 at its lowest. GREEN polarity WAS observed — at 9.4-11.6 (span 1.882) and in three stable runs (1.759/1.832/1.848) — but never beneath the shipped ceiling. The author states this honestly as an inference from data rather than a run they can paste, the reviewer confirmed the inference is sound (quiescence is strictly more favourable). Sound inference is still not observed run. WHY THAT MATTERS (§11.4.226). A registered, topology-preserving guard that never executes is indistinguishable from a guard that would fail if it did. Nothing currently ensures it ever runs: no freshness contract, no scheduled quiet-window invocation, no tracked item — which is precisely the unexecuted-standing-guard class §11.4 names, where registration gets treated as coverage while execution is nobody's job. The forensic precedent in that anchor is two standing guards that, when finally executed, immediately emitted latent FAILs. ACCEPTANCE. (a) Execute both quiescence-gated timing tests on a host whose 1-minute loadavg is genuinely < 0.75/cpu — no agent fleet, no concurrent build — and capture the run as an artifact showing the resolved load reading alongside the measured span exponential. the precondition is provable from the artifact and not asserted. (b) Record the observed GREEN polarity in the item as in the test source, replacing the current honest-inference note. (c) If the gate FAILS under genuine quiescence, that is the finding this item exists to surface, and reopens the threshold-calibration question rather than being worked around. (d)

ATM ID	Type	Status	Severity	Description
				Establish a freshness contract per §11.4.226: how stale a quiescent verdict may be before the gate counts as uncovered again, so this does not silently return to never-executing. HONEST BOUNDARY. This item does not claim the gate is wrong. The evidence points the other way — quiescence is strictly more favourable than the loads where GREEN was already observed. It claims only that the run has not happened, and that an unexecuted guard cannot be cited as coverage. PROVENANCE. Raised as IMPORTANT-2 by the independent Fabric substrate reviewer of the BOB-109 work, and independently corroborated: the reviewer verified the SKIP is loud and prints its resolved numbers, and confirmed the shipped tests do SKIP at today's load.